

Case 12: Negative — Wrong HTTP Method

Intermediate

Verify the API rejects requests made with the wrong HTTP method.

New concept: Testing wrong HTTP methods on endpoints

API Endpoint

POST `https://jsonplaceholder.typicode.com/posts/1`

What You Will Learn

- HTTP method restrictions per endpoint
- 405 Method Not Allowed status code
- Difference between `/posts` (collection) and `/posts/1` (single resource)
- Parametrizing multiple wrong methods in one test

Study Plan

Step 1 — Method restrictions

Not every HTTP method works on every endpoint. POST `/posts/1` makes no sense — you can't create a new resource at a specific ID. A real API returns 405 Method Not Allowed.

Step 2 — Collection vs single resource

`/posts` is a collection — supports GET (list) and POST (create). `/posts/1` is a single resource — supports GET, PUT, PATCH, DELETE. Mixing these up is a common mistake.

Step 3 — How to send different methods

You already know `requests.get()`, `requests.post()`, `requests.put()`, `requests.patch()`, `requests.delete()`. Think: how would you parametrize the method itself?

Step 4 — Parametrizing the method

Instead of parametrizing a value, you can parametrize the `requests` function itself. Think: what would each parametrize entry look like if the changing thing is the method?

Step 5 — Think about the loop

Each parametrize iteration gets one method. Do you need a for loop inside the test, or does `parametrize` handle all the repetition?

Think Before You Code

Ask yourself these questions before writing a single line:

- What is the changing value between each test run — the URL, the payload, or the method?
- Can you parametrize a function itself, not just a value?
- Do you need a for loop inside the test?
- What HTTP status code means 'wrong method used'?
- What is the difference between `/posts` and `/posts/1` in terms of allowed methods?

Your Task

Write a parametrized pytest test that sends wrong HTTP methods to `/posts/1`.

Test these methods on `/posts/1` (which doesn't support them as 'wrong' use):

- `requests.post` (POST to a specific resource makes no sense)

And these methods on `/posts` (collection) that don't belong:

- `requests.delete` (DELETE on a collection)
- `requests.put` (PUT on a collection)

Before writing:

- What is the `parametrize` variable here?
- Can you pass a function (`requests.post`) as a `parametrize` value?
- Do you need a loop inside the test?

The test must verify:

1. Use `parametrize` for the methods
2. Assert status is 404 or 405 (mark as `xfail` — `JSONPlaceholder` is lenient)

BONUS: Print the status code for each method — what does `JSONPlaceholder` actually return?

Hint: `@pytest.mark.parametrize('method', [requests.post, requests.delete, requests.put])` — then call `method(url)` inside the test.